

Indigenous Starchy Food Classifier — Training Notebook

This notebook trains an image classifier to distinguish between visually similar indigenous starchy food types, using transfer learning on a pretrained CNN.

How to use this notebook:

1. Run each cell in order (Shift+Enter), top to bottom.
2. Read the short note above each cell before running it — a few cells need you to fill something in (marked with `# >>> EDIT THIS`).
3. If a cell errors, stop and share the error message before continuing — don't skip ahead.

Before you start: In Colab, go to `Runtime > Change runtime type` and set Hardware accelerator to **GPU** (T4 is fine). This makes training much faster than CPU.

Step 1 — Check GPU is available

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

if device.type != "cuda":
    print("\nWARNING: No GPU detected. Go to Runtime > Change runtime type "
          "and select GPU, then re-run this cell.")
```

Using device: cuda

Step 2 — Install/confirm required packages

Colab already has PyTorch and torchvision installed, so this just confirms versions and installs a couple of extras we need for evaluation plots.

```
!pip install -q scikit-learn seaborn

import torch
import torchvision
print("torch:", torch.__version__)
print("torchvision:", torchvision.__version__)
```

torch: 2.11.0+cu128
torchvision: 0.26.0+cu128

Step 3 — Upload your dataset

Your dataset should be the `dataset` folder created by `split_dataset.py` on your computer, with this structure:

```
dataset/
  train/
    Amala/
    Eba/
    Semo/
    Pounded Yam/
  val/
    ...
  test/
    ...
```

On your computer: right-click the `dataset` folder → Send to → Compressed (zipped) folder. This creates `dataset.zip`.

Then run the cell below — it will pop up an upload button. Select your `dataset.zip`. (For a few hundred MB this may take a few minutes depending on your connection.)

If you'd rather use Google Drive instead of re-uploading each time, let me know and I'll swap this cell for a Drive-mount version.

```

from google.colab import files
import zipfile
import os

uploaded = files.upload() # select dataset.zip when prompted

zip_name = list(uploaded.keys())[0]
extract_dir = "/content/dataset"

with zipfile.ZipFile(zip_name, "r") as zip_ref:
    zip_ref.extractall("/content")

# Handle the case where the zip contains a nested 'dataset' folder vs files at top level
if not os.path.isdir(extract_dir):
    # find the extracted folder name automatically
    extracted_items = [d for d in os.listdir("/content") if os.path.isdir(f"/content/{d}")]
    print("Extracted folders found:", extracted_items)
    print("If 'dataset' isn't among them, update DATASET_DIR in the next cell manually.")

print("\nDone extracting.")

```

Choose Files dataset_v2_resized.zip

dataset_v2_resized.zip(application/x-zip-compressed) - 37303637 bytes, last modified: 7/28/2026 - 100% done
 Saving dataset_v2_resized.zip to dataset_v2_resized.zip
 Extracted folders found: ['.config', 'dataset_v2_resized', 'sample_data']
 If 'dataset' isn't among them, update DATASET_DIR in the next cell manually.

Done extracting.

```

DATASET_DIR = "/content/dataset_v2_resized"
import os
for split in ["train", "val", "test"]:
    split_path = os.path.join(DATASET_DIR, split)
    if os.path.isdir(split_path):
        categories = sorted(os.listdir(split_path))
        print(f"{split}: {categories}")
    else:
        print(f"WARNING: {split_path} not found.")

```

```

train: ['Amala', 'Eba', 'Pounded Yam', 'Semo']
val: ['Amala', 'Eba', 'Pounded Yam', 'Semo']
test: ['Amala', 'Eba', 'Pounded Yam', 'Semo']

```

✓ Step 4 — Define data transforms and loaders

Training images get augmented (random flips, rotation, color jitter) so the model doesn't just memorize exact photos. Validation and test images are only resized and normalized, since we want to evaluate on realistic, unaltered images.

```

import os, shutil

for split in ["train", "val", "test"]:
    split_path = f"{DATASET_DIR}/{split}"
    for cat in os.listdir(split_path):
        cat_path = f"{split_path}/{cat}"
        if os.path.isdir(cat_path):
            images = [f for f in os.listdir(cat_path)
                      if f.lower().endswith((".jpg", ".jpeg", ".png", ".bmp", ".gif", ".webp"))]
            if len(images) == 0:
                print(f"Removing empty/invalid folder: {split}/{cat}")
                shutil.rmtree(cat_path)

```

```

from torchvision import transforms, datasets
from torch.utils.data import DataLoader

IMG_SIZE = 224
BATCH_SIZE = 32 # >>> EDIT THIS to 16 if you get an out-of-memory error

train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
])

```

```
eval_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

train_dataset = datasets.ImageFolder(f"{DATASET_DIR}/train", transform=train_transform)
val_dataset = datasets.ImageFolder(f"{DATASET_DIR}/val", transform=eval_transform)
test_dataset = datasets.ImageFolder(f"{DATASET_DIR}/test", transform=eval_transform)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

class_names = train_dataset.classes
print("Categories:", class_names)
print("Train images:", len(train_dataset))
print("Val images:", len(val_dataset))
print("Test images:", len(test_dataset))
```

```
Categories: ['Amala', 'Eba', 'Pounded Yam', 'Semo']
Train images: 891
Val images: 189
Test images: 195
```

✓ Step 5 — Sanity check: look at a batch of training images

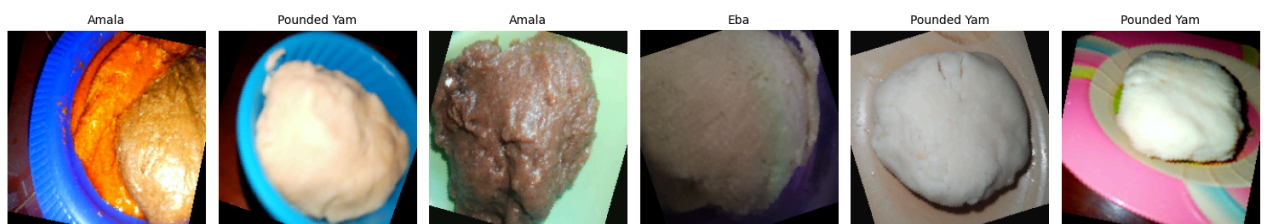
Quick visual check that images and labels loaded correctly and augmentation looks reasonable (not so extreme the food is unrecognizable).

```
import matplotlib.pyplot as plt
import numpy as np

def unnormalize(img_tensor):
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img = img_tensor.numpy().transpose((1, 2, 0))
    img = std * img + mean
    return np.clip(img, 0, 1)

images, labels = next(iter(train_loader))

fig, axes = plt.subplots(1, 6, figsize=(15, 4))
for i, ax in enumerate(axes):
    ax.imshow(unnormalize(images[i]))
    ax.set_title(class_names[labels[i]], fontsize=10)
    ax.axis("off")
plt.tight_layout()
plt.show()
```



✓ Step 6 — Build the model (transfer learning)

We use EfficientNet-B0 pretrained on ImageNet as the backbone, and replace its final classification layer with one sized for our categories. The backbone's early layers are frozen at first (they already know general visual features); only the new classifier head trains initially.

```
from torchvision import models
import torch.nn as nn
```

```

num_classes = len(class_names)

model = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)

# Freeze the pretrained feature extractor
for param in model.features.parameters():
    param.requires_grad = False

# Replace the classifier head for our number of categories
in_features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(in_features, num_classes)

model = model.to(device)
print(model.classifier)

```

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwightman-7f5810bc.pth [00:00<00:00, 189MB/s]

100%|██████████| 20.5M/20.5M [00:00<00:00, 189MB/s]

Sequential(

- (0): Dropout(p=0.2, inplace=True)
- (1): Linear(in_features=1280, out_features=4, bias=True)

)

✓ Step 7 — Train the model

This trains just the new classifier head for a number of epochs, tracking training and validation loss/accuracy so we can see if the model is learning well or overfitting.

If you get a CUDA out-of-memory error, reduce `BATCH_SIZE` in Step 4 and re-run from there.

```

import torch.optim as optim
import time

NUM_EPOCHS = 15 # >>> EDIT THIS - increase if accuracy is still improving at the end

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.classifier.parameters(), lr=0.001)

history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

for epoch in range(NUM_EPOCHS):
    start = time.time()

    # --- training ---
    model.train()
    running_loss, running_correct, total = 0.0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)
        running_correct += (outputs.argmax(1) == labels).sum().item()
        total += labels.size(0)

    train_loss = running_loss / total
    train_acc = running_correct / total

    # --- validation ---
    model.eval()
    val_running_loss, val_running_correct, val_total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            val_running_loss += loss.item() * images.size(0)
            val_running_correct += (outputs.argmax(1) == labels).sum().item()
            val_total += labels.size(0)

    val_loss = val_running_loss / val_total
    val_acc = val_running_correct / val_total

    history["train_loss"].append(train_loss)

```

```

history["train_acc"].append(train_acc)
history["val_loss"].append(val_loss)
history["val_acc"].append(val_acc)

elapsed = time.time() - start
print(f"Epoch {epoch+1}/{NUM_EPOCHS} ({elapsed:.0f}s) "
      f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} "
      f"val_loss={val_loss:.4f} val_acc={val_acc:.4f}")

```

```

Epoch 1/15 (7s) train_loss=1.0430 train_acc=0.6442 val_loss=0.7829 val_acc=0.8095
Epoch 2/15 (7s) train_loss=0.6490 train_acc=0.8395 val_loss=0.6267 val_acc=0.8360
Epoch 3/15 (6s) train_loss=0.5155 train_acc=0.8676 val_loss=0.4991 val_acc=0.8730
Epoch 4/15 (7s) train_loss=0.4367 train_acc=0.8833 val_loss=0.4577 val_acc=0.8783
Epoch 5/15 (6s) train_loss=0.4184 train_acc=0.8765 val_loss=0.4138 val_acc=0.8836
Epoch 6/15 (7s) train_loss=0.3505 train_acc=0.9035 val_loss=0.3862 val_acc=0.9048
Epoch 7/15 (6s) train_loss=0.3309 train_acc=0.9035 val_loss=0.3508 val_acc=0.9153
Epoch 8/15 (7s) train_loss=0.3190 train_acc=0.9046 val_loss=0.3521 val_acc=0.9048
Epoch 9/15 (6s) train_loss=0.2949 train_acc=0.9091 val_loss=0.3271 val_acc=0.9101
Epoch 10/15 (7s) train_loss=0.2972 train_acc=0.9203 val_loss=0.3083 val_acc=0.9206
Epoch 11/15 (6s) train_loss=0.2715 train_acc=0.9282 val_loss=0.3169 val_acc=0.9259
Epoch 12/15 (7s) train_loss=0.2720 train_acc=0.9259 val_loss=0.2993 val_acc=0.9153
Epoch 13/15 (6s) train_loss=0.2545 train_acc=0.9304 val_loss=0.2744 val_acc=0.9206
Epoch 14/15 (7s) train_loss=0.2446 train_acc=0.9416 val_loss=0.2745 val_acc=0.9048
Epoch 15/15 (6s) train_loss=0.2371 train_acc=0.9282 val_loss=0.2819 val_acc=0.9259

```

Step 8 — Plot training curves

If validation accuracy plateaus or drops while training accuracy keeps climbing, that's a sign of overfitting — worth flagging when we review results together.

```

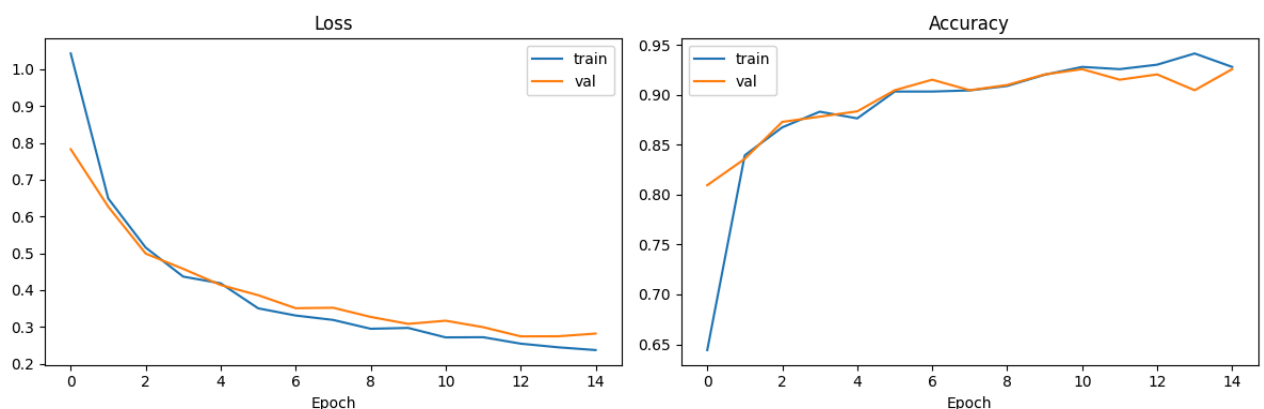
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history["train_loss"], label="train")
axes[0].plot(history["val_loss"], label="val")
axes[0].set_title("Loss")
axes[0].set_xlabel("Epoch")
axes[0].legend()

axes[1].plot(history["train_acc"], label="train")
axes[1].plot(history["val_acc"], label="val")
axes[1].set_title("Accuracy")
axes[1].set_xlabel("Epoch")
axes[1].legend()

plt.tight_layout()
plt.show()

```



Step 9 — Evaluate on the held-out test set

This is the real measure of how well the model generalizes: accuracy, per-class precision/recall, and a confusion matrix showing exactly which categories get mixed up with each other.

```

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

```

```

model.eval()
all_preds, all_labels = [], []

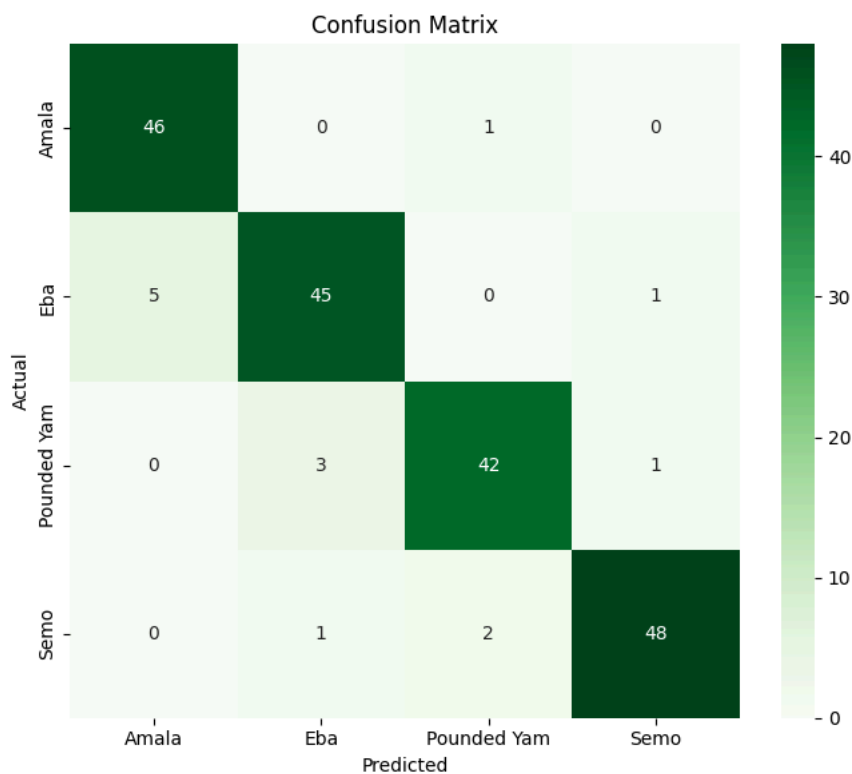
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        preds = outputs.argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(labels.numpy())

print(classification_report(all_labels, all_preds, target_names=class_names))

cm = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

```

	precision	recall	f1-score	support
Amala	0.90	0.98	0.94	47
Eba	0.92	0.88	0.90	51
Pounded Yam	0.93	0.91	0.92	46
Semo	0.96	0.94	0.95	51
accuracy			0.93	195
macro avg	0.93	0.93	0.93	195
weighted avg	0.93	0.93	0.93	195



Step 10 — Save the trained model

Saves the model weights plus the category list, so you (or I) can reload it later without retraining from scratch.

```

import json

torch.save(model.state_dict(), "food_classifier.pth")

with open("class_names.json", "w") as f:
    json.dump(class_names, f)

from google.colab import files

```

```
files.download("food_classifier.pth")  
files.download("class_names.json")  
  
print("Saved and downloaded: food_classifier.pth, class_names.json")
```

Saved and downloaded: food_classifier.pth, class_names.json

Next steps

- Review the classification report and confusion matrix above together — look especially at which categories (if any) get confused with each other.
- If a category is underperforming, it may need more/better training images rather than a code change.
- Once Pounded Yam (and any other categories) have enough labeled images, re-run `split_dataset.py` locally, re-zip and re-upload the dataset, and re-run this notebook from Step 3 onward.